SPAM SHIELD X

An Interim report Submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY

in

Computer Science & Engineering

by

Jalumuri Sreeja

2200030327

Under the esteemed guidance of

Mr. Sharath Chandra
H
R
Gaint Clout Technologies pvt ltd

DEPARTMENT OF BIOTECHNOLOGY K L UNIVERSITY

Green Fields, Vaddeswaram, Tadepalli, Guntur - 522 302, Andhra Pradesh.

2025-26

Gaint Clout Technologies pvt ltd
Orbit Knowledge City, Raidurg, Hyderabad, Telangana 500019

Bonafide Certificate

This is to certify that this Interim report entitled **"**SPAM SHIELD X**"** Submitted to **Department of Computer Science & Engineering, KL Deemed to be University, Guntur, in** connection with the University internship program is a bonafide record of work done by "**Jalumuri Sreeja"** under my supervision at the **"**Gaint Clout Technologies pvt ltd" from "14th July 2025" to "8th November 2025"

Mr. Sharath Chandra

HR

K L Deemed to be UNIVERSITY

DEPARTMENT OF Computer Science & Engineering

Certificate

This is Certified that the project entitled **"SPAM SHIELD X"** which is a experimental work carried out by "2200030327", in partial fulfilment for the award of the degree of Bachelor of Technology in Department of **Computer Science & Engineering**, during the year 2025-26. The project has been approved as it satisfies the academic requirements.

Dr. N. Ravendra	**Dr.T.Pavan Kumar**
Department Coordinator	**Head of the Department**
Dr.Ch.Venkateswara Rao	**External**
University Guide	

ACKNOWLEDGEMENT

Being an intern at **Gaint Clout Technologies Pvt. Ltd.** has been a truly remarkable and memorable experience in my university journey. This internship provided me with a wonderful opportunity to gain practical exposure to real-time working environments and to enhance my technical and professional skills. It was a privilege to work with experienced professionals who were always willing to share their knowledge, provide guidance, and support me throughout the project.

I take this valuable opportunity to extend my heartfelt gratitude to all those who guided and supported me during my internship and made this experience a successful one.

First and foremost, I would like to express my sincere thanks to **Mr. Sharath Chandra** for his continuous encouragement, insightful suggestions, and valuable guidance throughout my internship period. I am also deeply thankful to **Ms. Rohini, Ms. Nikitha and Mr. Elisha** from Gaint Clout Technologies Pvt. Ltd. for their constant support, supervision, and motivation, which helped me in understanding the project objectives and improving my technical knowledge.

I extend my profound gratitude to my university guide **Mr. CH. Venkateswar Rao** for his valuable guidance, support, and encouragement throughout the internship. His constructive feedback and valuable insights greatly contributed to my learning and professional development.

I also thank **Dr. B. Suresh** In-Charge of Practice School **and Dr.T.Pavan Kumar** Head of the Department of **K L University, Vijayawada**, for their constant encouragement and assistance in the successful completion of my internship.

Once again, I express my sincere appreciation to all those who directly or indirectly supported and guided me throughout this journey. The knowledge and experience gained during this internship will undoubtedly be beneficial for my future professional endeavors.

Jalumuri Sreeja

2200030327

ABSTRACT

The goal of the project “SPAM SHIELD X" is to create an intelligent email classification system that uses sender and content data to automatically filter incoming emails and identify possible spam. In order to securely retrieve emails, the system integrates with the Gmail API using OAuth authentication. These emails are then examined and divided into two primary categories: Spam (unwanted or suspicious messages like promotional offers or cash prize scams) and Inbox (legitimate emails like university communications).

Flask is used to build the system's backend, which offers RESTful APIs for reporting, classification, and email retrieval. An IP extraction and geolocation module uses services like ip-api to determine the sender's actual location, while a strong spam detection algorithm examines email content and metadata to guarantee correct classification. This raises awareness of cybersecurity and aids in tracking down spammers. In order to guarantee dependability and scalability, the system also has tools for error handling, logging, and edge case management.

The project also uses IP-based filtering logic to automatically ban emails from unapproved smart devices (like smartwatches) and only permit emails from authorized sources (such laptops, desktop computers, and mobile phones). The system's usability is further strengthened by extensive testing, pagination, and visualization elements including folder-wise analytics and country-wise spam counts.

All things considered, this project offers a comprehensive email filtering and spammer detection solution that improves inbox security, shows spam patterns, and offers a scalable backend architecture for potential future integration with cutting-edge machine learning models.

Table of Content

TITLE	Page No.
Cover Page	1-3
Acknowledgement	4
Abstract	5
Table of Contents	6
List of Figures	7
Introduction	8
Problem Statement	9
Literature Review	10-13
Objectives	14-16
Scopes & Limitations	17-20
Modules & feature	21-27
Implementation & File Structure	28-33
Files & Results	34-45
Discussion	46-48
Conclusion	49
References	50-51

List of Tables

S.No	Table	Page No
1	Figure 1	34
2	Figure 2	34
3	Figure 3	35
4	Figure 4	35
5	Figure 5	36
6	Figure 6	36
7	Figure 7	37
8	Figure 8	37
9	Figure 9	38
10	Figure 10	38
11	Figure 11	39
12	Figure 12	39
13	Figure 13	40
14	Figure 14	40
15	Figure 15	41
16	Figure 16	41
17	Figure 17	42
18	Figure 18	42
19	Figure 19	43
20	Figure 20	43
21	Figure 21	44
22	Figure 22	44
23	Figure 23	45

INTRODUCTION

In both professional and academic settings, email has emerged as a crucial communication tool. Email systems are a major tool used by universities to communicate with faculty and students about schedules, official announcements, and course updates. However, users are regularly inundated with unsolicited spam or phishing emails due to the proliferation of online platforms, advertisements, and cyber threats. In addition to clogging inboxes, these spam messages present significant security risks, including malware links, phishing scams, and data breaches. In order to guarantee that only authentic and pertinent emails make it to the user's inbox, it is imperative to implement an intelligent email classification system.

The main goal of this project is to create an email filtering system that uses Flask and is connected to the Gmail API. The system's objective is to automatically categorize incoming emails into two groups: spam (irrelevant or suspicious emails) and inbox (legitimate emails pertaining to the university). Furthermore, by examining the email sender's IP address, the project goes beyond conventional spam detection by locating and identifying possible spammers geographically. This improves email security and aids in tracking down the source of malicious emails.

A thorough method of spam control is offered by combining IP geolocation tracking, Gmail OAuth authentication, and machine learning-based spam classification. The project guarantees accuracy and transparency in email classification by integrating network-level analysis with content-based filtering. Additionally, it helps users take the required security precautions by identifying IP addresses linked to questionable activity, adding a preventive mechanism.

In the end, the project helps create a more intelligent and secure communication environment in addition to improving user safety. The potential for real-time email intelligence systems that can adjust to changing online threats and communication patterns is demonstrated by the solution's automatic detection, classification, and localization of spam sources.

PROBLEM STATEMENT

In today’s digital world, users receive hundreds of emails daily, out of which a significant percentage are spam or phishing messages. False positives, or legitimate emails labeled as spam, or false negatives, or spam emails getting into the inbox, are the results of traditional email systems' frequent inability to accurately detect new or changing forms of spam. This problem is even more serious for universities because official academic emails may be misplaced in spam, leading to missed deadlines, misunderstandings between staff and students, and communication breakdowns. Therefore, a clever filtering system that reliably separates acceptable university emails from unsolicited ones is desperately needed.

Finding the origin of spam emails is a significant additional concern. In order to conceal their identities and make it more difficult to track and block them, spammers commonly use proxy IPs or phony sender addresses. Users are unable to identify the source of questionable activity in the absence of IP tracing, which may result in recurrent spam attacks. Therefore, email security and accountability would be greatly improved by a system that can extract and analyze IP addresses from email headers to identify the geographic location of spammers.

Additionally, new security threats are brought about by the proliferation of smart devices, including smartwatches and email clients based on the Internet of Things. These devices are potential entry points for malicious traffic because they frequently connect over shaky or insecure networks. As a result, the project adds another restriction: only allowing email access from reliable IP sources (such as desktop, laptop, or mobile) while blocking suspicious smart device IPs. This guarantees that email correspondence stays on verified and authenticated devices.

In conclusion, the main issues are automatically determining whether incoming emails are spam or inbox, utilizing IP analysis to determine the sender's location, and limiting unauthorized device-based access. By resolving this issue, a dependable, intelligent email management system that can boost productivity, guarantee security, and preserve the integrity of university communications will be created.

LITERATURE REVIEW

1. Email Exchanges and the Spam Issue:

In both business and academic settings, electronic mail, or email, has emerged as a vital communication tool. But along with its broad use came a sharp increase in unsolicited and misleading messages, or spam. Spam accounts for more than 45% of all email traffic worldwide, according to Radicati Group (2024). Phishing links, phony offers, or malicious attachments intended to steal confidential information or spread malware are frequently included in these emails.

Spam detection is a crucial part of contemporary email management systems since it not only clogs inboxes but also causes productivity loss, security breaches, and bandwidth waste.

2. Traditional Methods for Detecting Spam:

Blacklists, whitelists, and keyword matching were examples of rule-based systems used in early spam filtering. Heuristic rules were used by well-known tools such as SpamAssassin to categorize emails. Although these techniques worked well for structured spam, they were not flexible enough and needed frequent rule revisions as spam strategies changed.

Following this, Bayesian filtering gained popularity after being introduced by Paul Graham in 2002. Word occurrence in spam and non-spam messages is used by Bayesian classifiers to calculate probabilities. The emergence of sophisticated obfuscation techniques, image-based spam, and context variation reduced their effectiveness despite their ease of use and interpretability.

3. Spam Filtering Using Machine Learning:

Spam filtering moved toward data-driven learning and automated pattern recognition as machine learning (ML) advanced. algorithms like:

- Naive Bayes (NB) : For text-based spam, Naive Bayes (NB) uses probabilistic reasoning, which is straightforward but efficient.
- Support Vector Machines (SVM) : These are robust against high-dimensional text features and use hyperplanes to distinguish between spam and non-spam.
- Random Forests (RF) and Decision Trees (DT): These are effective at handling big feature sets and capturing non-linear patterns.
- Logistic Regression (LR):Interpretable and effective for binary classification is logistic regression (LR).
- K-Nearest Neighbors (KNN) : Emails are categorized by K-Nearest Neighbors (KNN) according to how close they are to labeled examples.

Almeida et al. (2011) showed that SVM and NB were reliable for text-based filtering by achieving >95% accuracy on the Enron-Spam dataset.

4. Advanced Spam Detection with Deep Learning:

Recent developments use deep learning architectures for email content semantic understanding, including Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs).

Long Short-Term Memory (LSTM) models, for example, are better than traditional classifiers at identifying phishing and image-based spam because they can capture contextual dependencies in messages.

By extracting both sequential and local features, hybrid systems that combine CNN and LSTM have produced state-of-the-art results.

Additionally, contextual spam filtering has been investigated using Transformer-based models such as BERT and GPT variants, which allow for a better comprehension of intricate sentence structures and hidden spam triggers.

5. Combining Geolocation and IP Tracking:

Finding the origin of spam emails is crucial for cyber-forensic investigations in addition to content analysis. Email headers contain the “Received From” fields, which record the sender’s IP address. The approximate geographic location of the spammer can be tracked by extracting this IP.

IPs are mapped to country, region, city, latitude, and ISP details using popular APIs like ip-api, ipinfo, and MaxMind GeoIP.

Researchers discovered that IP-based tracking aids in the detection of automated botnets that send bulk spam and the identification of spam clusters (e.g., Kaspersky Labs, 2022).

Blocking recurring spam sources and improving accountability are two benefits of combining IP tracing and spam classification.

6. Using OAuth and the Gmail API to Provide Secure Email Access:

Google Cloud's Gmail API uses OAuth 2.0 authentication to provide safe programmatic access to Gmail mailboxes. This guarantees token-based controlled access and does away with the need to share passwords.

While adhering to Google's security guidelines, developers can automatically retrieve, read, label, and categorize emails.

Scoped access is supported by the API, which enables the system to read either the entire content or just the email metadata as needed. It enables a strong backend architecture for automated email analysis and classification when used in conjunction with Flask.

7. Using the Flask Framework for Backend Development:

A popular lightweight Python web framework for creating RESTful APIs is called Flask. It is perfect for combining natural language processing and machine learning models because of its modular design.

Flask is commonly utilized in spam detection systems to:

- Provide endpoints for the classification and analysis of emails.
- Control the OAuth token flow for the Gmail API.
- Take care of pagination, error correction, and logging for big datasets.
- In order to connect the frontend for real-time visualization, provide APIs.

Applications for spam detection can have a scalable architecture thanks to the combination of Flask and machine learning models.

8. Device-Based Filtering and Automation:

Contextual automation is crucial in contemporary systems. The proposed project's unique goal is to automatically block emails if the sender's IP originates from untrusted devices, like smartwatches or Internet of Things devices, which are frequently used to distribute spam.

Controlled communication and enhanced security are ensured by only accepting emails from approved devices (such as laptops, desktop computers, and mobile phones).

This idea of device-IP validation is a new trend in cybersecurity that follows the guidelines of zero-trust architecture, which requires that every connection be confirmed before being accepted.

9. Examining Related Systems and Any Gaps:

The primary functions of current spam detection tools, such as Barracuda, SpamAssassin, and Gmail's built-in filters, are content filtering and URL blacklisting. But they rarely include:

- Geolocation mapping at the IP level,
- Blocking techniques based on devices, and
- Integration of real-time Gmail OAuth in academic systems that are custom-built.

By integrating device-type validation, IP tracing, and email classification into a single automated process utilizing Flask and the Gmail API, this project fills in these gaps.

10. Results of the Literature:

It is clear from the review that:

- Accurate spam detection relies heavily on machine learning models.
- Semantic email analysis is improved by deep learning and natural language processing.
- An extra degree of assurance for identifying the source of spam is offered by IP-based tracing.
- Scalable, secure, and real-time deployment is made possible by the integration of the Flask and Gmail APIs.

Therefore, by combining device-based access control, IP location tracking, and spam detection, the suggested system advances the field and offers a comprehensive approach to intelligent email filtering.

OBJECTIVES

1. Configuring the Flask Backend Environment:
    - Set up the Flask project's infrastructure (app, routes, utils, etc.).
    - Set up the dependencies and virtual environment (requirements.txt).
    - Configure config.py to control API keys and environment variables.
2. Integration of the Gmail API with OAuth:
    - Using Google Cloud Console, create and configure your Gmail API credentials.
    - Put the OAuth flow into practice for safe user authentication.
    - Safely store and update access tokens for future use.
3. Gmail Token and Route Management:
    - Develop Flask routes to handle Gmail OAuth authorization and callback
    - Manage token validation, storage, and expiry checks
    - Enable Gmail revoke/logout functionality for security
4. Fetching and Storing Emails:
    - Use the /fetch_all_emails API to get Gmail emails.
    - Keep emails in memory or temporarily stored for categorization.
    - For effective email retrieval, use pagination.
5. System for Classifying Emails:
    - Examine the email's subject, content, and sender information.
    - Put emails pertaining to the university in the inbox.
    - Identify and mark emails that are fraudulent, promotional, or scams as spam.
    - Dynamically link folders to classification logic
6. Identification and Verification of Spam:
    - Create spam-check rules based on sender domain, pattern match, and keywords.
    - Verify the accuracy of spam detection using test datasets.
    - Create unit tests that address a variety of spam edge cases.
7. Extracting and Parsing IP Addresses:
    - Retrieve the sender's IP address from the email headers (X-Originating-IP, Received-SPF).
    - Clean and verify a variety of IP formats.
    - Address situations with missing or distorted headers.

8. IP Geolocation and Monitoring of Spammers:
    - Integrate IP geolocation lookup with ip-api.com.
    - Retrieve location information (ISP, city, region, and country).
    - Use fallback mechanisms to gracefully handle unsuccessful lookups.
9. IP Blocking Logic for Smart Devices:
    - Determine whether the email is coming from an IoT source or a smartwatch IP.
    - Automatically prevent such IPs from getting any more emails.
    - Only accept emails from desktop, laptop, or mobile IP addresses.
10. Error Handling and Logging:
    - Every significant action (OAuth, fetch, classify, IP lookup) should have logging implemented.
    - For debugging, record warnings, errors, and system data.
    - Include Flask error handlers for null, 500, and 404 responses.
11. Reporting and Data Analytics:
    - Count emails in each folder (spam vs. inbox).
    - List the top 5 nations where spam is most common.
    - Provide a structured JSON response that summarizes email statistics.
12. Design and Documentation of APIs:
    - Establish distinct API endpoints for each functionality (/stats, /fetch, and /classify).
    - Use Markdown (README.md) or Swagger UI to document endpoints.
    - Provide examples of requests and responses as well as the authentication process.
13. Assurance of Quality and Testing:
    - Execute integration tests from OAuth → fetch → classify → response.
    - Create more than 20 dummy emails to test performance.
    - Verify the accuracy of spam, IP blocking, and folder assignment consistency.
14. Code Refactoring and Optimization:
    - Remove any duplicate or superfluous code segments.
    - Move reusable functions to utility modules (utils.py)
    - Make sure the logging format is consistent and PEP8 is followed.

15. Project documentation and final deliverables:
    - Make an architecture diagram and an ER diagram.
    - Create a thorough README.md document that includes setup, usage, and output examples.

SCOPES & LIMITATIONS

1. SCOPES:

This project's primary objective is to use Flask as the backend framework and Gmail API integration to automatically detect and categorize incoming emails as either spam (malicious/unwanted) or inbox (legitimate). To further improve knowledge of spam sources, the system also carries out IP extraction and geolocation mapping to track the origin of the spammer. Additionally, the project aims to prevent spam emails from coming from unapproved devices (like smartwatches) while permitting emails from reliable sources like desktops, laptops, and cell phones.

Functional Range:

1.1. Classification of Emails:

- The system uses OAuth-based authentication to retrieve emails from the Gmail inbox.
- Every email is automatically examined and classified as either spam or inbox.
- Finding keywords, sender domains, and questionable patterns like "cash prize," "lottery," "promotion," etc. are all part of the classification logic.

1.2. Identifying Spammers by IP Address:

- The IP address is extracted from the email headers by the project.
- The ip-api service is used to process the IP address and extract geolocation information (Country, Region, City, Latitude, Longitude, and ISP).
- This aids in identifying the spam email's source.

1.3. Access Restrictions Based on Devices:

- Only emails from trusted IP devices (desktop, laptop, and mobile) are permitted by the system.
- Any email that is found to be coming from unapproved devices—such as smartwatches—is automatically tagged and placed in the Spam folder.

1.4. Gmail API Integration:

- OAuth 2.0 authentication allows for a secure connection with Gmail.
- Users can retrieve, examine, and categorize every Gmail email thanks to the project.
- Routes for the classification endpoints and /fetch_all_emails were implemented.

1.5. Analysis and Visualization of Data:

- Gives email counts by folder (inbox vs. spam).
- Shows the top five countries for spammers by country.
- Improvements to pagination and API response for effective email management.

1.6. Managing Errors and Recording Them:

- Complete error handling is implemented for IP lookups, API responses, and Gmail authentication.
- Keeps thorough records in order to track and debug spam analysis activities.

1.7. Validation and Testing:

- 20+ dummy emails were used for testing in order to confirm the accuracy of the IP location and spam classification.
- Token management, Gmail OAuth flow, and error scenarios (invalid/expired tokens) were all verified.
- Made sure that the geolocation mapping and folder assignment were correct.

1.8. Deliverables for the project:

- Flask backend application that is fully operational.
- Gmail API configuration using legitimate OAuth credentials.
- Integration of IP geolocation services with ip-api.
- Documentation such as architecture diagrams, ER diagrams, and API documentation.
- Complete the README and test cases for the project.

2. LIMITATIONS:

Although the project accomplishes its main goals, it has some practical and technical limitations, which are listed below:

2.1. Exclusive to the Gmail Platform:

- Only the Gmail API is integrated into the system; emails from other providers, like Yahoo and Outlook, cannot be retrieved or categorized.

2.2. Reliance on OAuth and the Gmail API:

- For the application to function, valid OAuth tokens and API credentials are needed.
- The system's performance may be impacted by any modifications made to Gmail's permissions or API structure.

2.3. Detecting Spam Using Keywords:

- Predefined keyword matching and content heuristics are the mainstays of spam classification.
- For contextual understanding, it doesn't use sophisticated AI/ML models like deep learning or natural language processing.
- For borderline cases, false positives or negatives could happen.

2.4. Limitation on IP Address Accuracy:

- The availability of IP data determines the accuracy of the ip-api geolocation.
- In certain situations (VPNs, proxies, cloud servers), the detected IP might not be the spammer's actual location.

2.5. Gmail does not have real-time blocking:

- Although the system can detect and categorize spam, it is unable to stop Gmail from receiving emails from the same IP in the future.
- Blocking makes sense in the Flask system; it has no effect on Gmail's built-in controls.

2.6. IP identification of smartwatches:

- IP and user-agent pattern detection are used to identify devices.
- A smartwatch may evade detection if it imitates a desktop or mobile IP header.

2.7. No Frontend Integration Scope in This Document:

- Only backend functionality and APIs are covered in this documentation.
- The backend scope here does not include frontend UI or visualization logic, even though frontend (React + Vite) may exist.

2.8. Validation Guidelines for Static Devices:

- The list of permitted devices (desktop, laptop, and mobile) is manually created and is unable to recognize or update itself.

2.9. Restricted Scalability:

- The current system is intended for small-scale or individual use.
- It is not designed to handle thousands of emails every day at an enterprise level.

2.10. Testing Restrictions on Datasets:

- A small dataset of both real and dummy emails was used for validation.
- For universal accuracy, more extensive testing across various Gmail accounts and spam types might be needed.

2.11. Dependency on a network:

- IP lookups and Gmail API calls require consistent internet access.
- There is no support for offline analysis.

2.12. Boundaries of Permission and Privacy:

- The program needs read-only access to a Gmail account.
- Although it closely adheres to OAuth scopes, it is unable to edit or remove emails in Gmail.

2.13. Absence of adaptive learning or machine learning:

- The rules used for spam detection are non-adaptive and rule-based.
- Accurate classification cannot be learned or improved over time by the system.

2.14. Data storage and logging:

- There is no cloud-based centralized monitoring in place; logs are kept locally.
- Logs may be erased in the event of a server restart unless they are externally backed up.

MODULES & FEATURE

1. Initialization of the Flask Backend:

1.1. Objective:

Construct the backend framework for Gmail integration and API communication.

1.2. Features:

- Set up routes and initialize the Flask application.
- Gmail, classifier, IP parser, geolocation, and database (if any) should all be connected.
- For frontend communication, enable CORS.
- Include server startup checks for Gmail API settings and credentials.
- To confirm the backend's health, define and test the route.

2. Gmail OAuth Authentication:

2.1. Objective:

Authorize and securely connect to the Gmail account.

2.2. Features:

- Create OAuth 2.0 credentials (Client ID, Secret, Redirect URI) using Google Cloud Console.
- Put the Gmail login process into practice:
  - Send the user to the Google Consent page.
  - Get your authorization code, then trade it in for tokens.
- Tokens for API access and refresh should be safely stored.
- Tokens are automatically renewed when they expire.
- When a user disconnects, log them out or revoke their access to Gmail.
- Address authorization issues (missing permissions, invalid scopes, and expired tokens).

3. Email Fetching through the Gmail API:

3.1. Objective:

The objective is to retrieve emails for classification and analysis.

3.2. Features:

- To retrieve email content and metadata (subject, from, headers), use the Gmail API.
- Assist with retrieving:
  - Email inbox.
  - Emails that are spam.
  - If necessary, create custom queries.
- To obtain all fetched data, implement the /fetch_all_emails endpoint.
- For large inboxes, add pagination and limit parameters.
- Manage pagination tokens and API quota limits.
- Verify the email's data integrity (valid sender, non-null fields).

4. Email Categorization (Spam/Inbox):

4.1. Objective:

Examine every email to determine whether it is spam or inbox content.

4.2. Features:

- Describe classification based on rules:
  - Domain names from universities → Inbox (e.g., @college.ac.in,.edu).
  - Spam indicators (such as "win," "offer," "cash prize," and "lottery") → Spam.
- Examine email headers and sender reputation.
- If available, include the authentication results (SPF/DKIM checks).
- Give it a spam score and classify it appropriately.
- For every email, return the folder type (spam or inbox).
- Note the classification reasons (e.g., domain mismatch, keyword hit).
- Link Flask routes to display folders and classification logic.

5. Validation of Spam and Test Cases:

5.1. Objective:

Make sure spam detection is accurate.

5.2. Features:

- Make several test cases with fictitious data (20+ emails).
- Verify the accuracy of the classifier using:
  - Test subject (spam keywords).
  - Domain test (unknown vs. permitted).
  - IP test (suspicious vs. trusted IP).
- Record the classification outcomes for every test email.
- Keep track of false positives and adjust thresholds.
- Use unit tests to automate validation.

6. Parsing Email Headers:

6.1. Objective:

The objective is to extract technical metadata from raw Gmail emails.

6.2. Features:

- To determine the origins of IP addresses, parse "Received" headers.
- From, To, Subject, Return-Path, and Date are the fields that need to be extracted.
- Find several hops in the mail relay path (received lines).
- Ignore private IPs (such as 192.168.x.x) and filter legitimate public IPs.
- Enhance the parsing logic to deal with encoded data or malformed headers.
- For tracking, send the extracted IPs to the IP Geolocation module.

7. Validation and IP Parser:

7.1. Objective:

The objective is to reliably extract and validate the sender's IP address from email headers.

7.2. Features:

- Use the bottom-most valid Received header to extract the IP address.
- Take out the 10.x, 127.x, and 192.168.x local/loopback IPs.
- Verify the IPv4 and IPv6 formats.
- If the email was routed through more than one relay, identify the various IP addresses.
- Make sure that for spammer tracing, only one final valid IP address is chosen.
- For reference, record every parsed IP address.

8. IP geolocation (integration of IP-api):

8.1. Objective:

Determine the geographic origin of the spammer.

8.2. Features:

- Use the extracted IP to establish a connection to the ip-api service.
- Get location information:
  - Country
  - City
  - Region
  - Internet service provider, or ISP
- Adapt gracefully to failed lookups and API rate limits.
- Use caching (optional) to stop the same IPs from being looked up repeatedly.
- Create reports like the Top 5 countries for spammers.
- Combine IP information with the classification outcomes.

9. IP Filtering for Smart Devices:

9.1. Objective:

Mark or block emails coming from unapproved devices (like smart watches).

9.2. Features:

- Determine the type of device using the User-Agent (if available) or IP/ISP patterns.
- Make logic that only permits laptop, desktop, and mobile sources.
- Block or automatically move unauthorized device-originated emails to spam.
- Keep a whitelist of secure devices up to date.
- Use sample IP scenarios to log and test blocking functionality.

10. Reporting and Analytics:

10.1. Objective:

Produce insightful information regarding classified emails.

10.2. Features:

- Email count by folder (spam vs. inbox).
- Spam count by country (Top 5).
- Time-based synopsis (optional).
- Provide frontend dashboard analytics data through an API.
- Combine spam patterns (repeated IPs, frequent domains, etc.).
- If necessary, visualize data for reporting or presentation.

11. Flask Routes (API Layer):

11.1. Objective:

Supply front-end and testing endpoints.

11.2. Features:

- /authorize: Start the Gmail OAuth process.
- /oauth2callback: Manage callbacks for authorization.
- /fetch_all_emails Get and examine Gmail correspondence.
- /classify - Return folder_type after classifying emails.
- /analytics: Return folder counts and countries with the highest rates of spam.
- Removing the Gmail access token is done with /logout.
- Provide standardized JSON answers along with the appropriate HTTP code.

12. Managing Errors and Recording Them:

12.1. Objective:

Make sure backend operations run smoothly and make troubleshooting simple.

12.2. Features:

- For Flask routes, add global exception handling.
- Record typical mistakes such as:
  - 404 (Route not found)
  - 400 (unsatisfactory request)
  - 401 (Auth failed)
  - 500 (errors on the server)
- Note the status and timestamp of every API call.
- Log summaries of spam detection.
- Don't log private information like user data or tokens.

13. Management of Configurations:

13.1. Objective:

Cleanly manage security and environment variables.

13.2. Features:

- To manage settings, use a configuration file (such as config.py).
- Store:
  - Client ID and Client Secret for Gmail.
  - Redirect web addresses.
  - API endpoints.
  - Spam score thresholds.
- Steer clear of hardcoding secrets in code.
- Support both development and production modes.

14. Testing and the Creation of Dummy Data:

14.1. Objective:

Verify the backend's complete functionality.

14.2. Features:

- Make fake inbox and spam emails (20+).
- Test:
  - Gmail fetch flow.
  - Output of classification.
  - Location lookup and IP extraction.
- Test the edge cases of Gmail authentication (missing consent, expired token).
- Use IPs similar to smartwatches to test IP blocking logic.
- Check analytics endpoints and folder counts.
- Execute the last integration test: OAuth → Fetch → Classify → Show outcomes.

15. Final Deliverables and Documentation:

15.1. Objective:

Get ready to submit or deploy the project.

15.2. Features:

- Create a README.md file with setup guidelines, environment setup, and usage instructions.
- Include API documentation in Markdown or Swagger formats.
- Record the Gmail integration process (either visually or in a step-by-step document).
- Create an ER Diagram for the database entities.
- Make an architecture diagram that illustrates how the IP API, Flask, Gmail API, and frontend interact.
- Provide screenshots of successful runs and test results.

IMPLEMENTATION & FILE STRUCTURE

1. Implementation Overview

1.1. Flask Project Configuration:

- Purpose:

  Establishing the Flask backend and acting as the project entry point is the goal.

- Execution:
  - Run.py launches the development server and initializes the Flask application.
  - The Flask application is set up, routes are imported, and required modules are connected by the app/__init__.py file.
  - For sensitive credentials, environment variables are loaded from.env.
  - App/config.py handles configurations such as Gmail API keys, credentials, and app settings.

1.2. Integration of the Gmail API (OAuth 2.0 Flow):

- Purpose:

  Use OAuth to authenticate with Gmail for safe email access.

- Execution:
  - The credentials.json file for Gmail is kept in app/.
  - Gmail_service.py oversees:
    - OAuth configuration
    - Exchange of Gmail tokens
    - Using the Gmail API to retrieve emails (messages.list, messages.get)
  - Flask routes manage token flow, callback, and authorization.
  - Tokens are automatically renewed when they expire and are safely stored.

1.3. Email Categorization (Spam/Inbox):

- Purpose:

  Determine if an email is spam or an official university message.

- Execution:
  - The logic in spam_filter.py allows for analysis of:
    - Sender domain and subject (e.g.,.edu = Inbox)
    - Keyword spam ("cash," "win," "offer," etc.)
    - Faulty authentication or dubious links.
  - Determines the folder type and produces a spam score:
    - Score < 50 → Inbox
    - Score ≥ 50 = Spam
  - Uses dummy data in test cases to validate results.

1.4. Geolocation and IP Extraction:

- Purpose:

  Use the IP address in the email header to determine the sender's location.

- Execution:
  - IP_locator.py:
    - Sender IPs are extracted from "Received" headers.
    - Removes local and private IPs and verifies the IP format.
    - Retrieves geolocation (country, city, and ISP) by calling the IP API.
  - Logs or fallback are used to gracefully handle unsuccessful lookups.
  - Analytics (top spam origins) later use country information.

1.5. IP blocking for smart devices (security filtering):

- Purpose:

  Limit the acceptance of emails from unapproved device IPs, such as smartwatches.

- Execution:
  - Security_utils.py verifies the IP source and device type.
  - Checks IP patterns against a whitelist (desktop, laptop, and mobile).
  - The blocked_ips.json file is updated with suspicious IPs.
  - These emails are automatically blocked or flagged.

1.6. Error handling and logging:

- Purpose:

  Preserve traceability, dependability, and debugging simplicity.

- Execution:
  - Logging is set up globally in utils.py.
  - Records analysis, IP lookups, OAuth success/failures, and classification decisions.
  - Routes.py has been updated to handle 404, 400, and null response errors.
  - Both the logging module and print() are used in debugging.

1.7. Testing and Dummy Data:

- Purpose:

  Verify IP parsing and spam logic.

- Execution:
  - More than 20 sample emails covering both inbox and spam scenarios are produced by dummy_data.py.
  - Every test email imitates the headers of actual Gmail messages.
  - Spam cases are used to test for IP mismatches, forged headers, and keywords.
  - No false positives are guaranteed in valid cases (university mails).

1.8. Routes & Endpoints for APIs:

- Purpose:

  Define Flask routes in order to facilitate front-end communication.

- Execution (in routes.py):
  - /authorize: Initiate the OAuth procedure.
  - /fetch_all_emails Retrieve and examine every Gmail message.
  - Classify a single email with the /classify_email command.
  - Get Inbox vs. Spam statistics at /analytics/folder_counts.
  - Get the top 5 countries where spam originates by visiting /analytics/top_countries.
  - Log out of your Gmail account with /revoke.
  - returns JSON data with pagination support.

1.9. Reporting and Analytics:

- Purpose:

  Produce information about spam versus inboxes and the sources of spam.

- Execution:
  - Within the fetch or analysis API, folder-wise email count logic is implemented.
  - IP locations are aggregated by country-wise spam analytics.
  - Data formatted for the front-end dashboard (visualization of the map and count).

1.10. Management of Configuration and Environment:

- Purpose:

  The purpose is to centralize all configuration data.

- Execution:
  - Gmail Client IDs, API URLs, spam thresholds, and logging modes are all managed by config.py.
  - Sensitive credentials are stored in.env files (not committed to GitHub).
  - Guarantees environment-specific configurations for both development and production.

1.11. Documentation for APIs:

- Purpose:

  The purpose is to give each backend endpoint a clear reference.

- Execution:
  - API routes, parameters, and sample responses are defined in swagger.json.
  - Response formats, endpoints, and usage are described in the markdown documentation (docs/API_DOCS.md).
  - All set to be imported into Postman or Swagger UI.

1.12. Final Testing and Integration:

- Purpose:

  Complete system validation.

- Execution:
  - Complete flow: Gmail OAuth → Emails fetched → Classification → IP location → Outcomes.
  - Tested using both real and fake Gmail data.
  - Logs for every step were confirmed.
  - Final cleanup and bug fixes were completed.
  - Results were checked against the anticipated results.

1.13. Deliverables and Documentation:

- Purpose:

  Get ready to submit the project.

- Execution:
  - README.md: Detailed instructions for installation and use.
  - The architecture and Gmail integration flow are described in detail in docs/docs.md.
  - The relationship between email, IP, and analysis results is shown in the ER Diagram.
  - The architecture diagram illustrates the interactions between the modules (OAuth → Fetch → Classify → IP Locate).

2. File Structure:

```
email_filter_project/
├── app/
│   ├── __init__.py
│   ├── blocked_ips.json
│   ├── config.py
│   ├── credentials.json
│   ├── dummy_data.py
│   ├── gmail_service.py
│   ├── ip_locator.py
│   ├── routes.py
```

```
│   ├── security_utils.py
│   ├── spam_filter.py
│   └── utils.py
├── dist/
│   ├── assets/
│   ├── index.html
│   └── vite.svg
├── docs/
│   ├── API_DOCS.md
│   ├── docs.md
│   ├── GMAIL_FLOW.md
│   └── README.md
├── scripts/
│   └── run_full_integration.sh
├── tests/
│   ├── test_gamil_integration.py
│   ├── test_ip_locator.py
│   ├── test_routes.py
│   └── test_spam_filter.py
├── tools/
│   └── run_dummy_classify.py
├── venv/
├── .env
├── requirements.txt
├── run.py
    └── swagger.json
```

FILES & RESULTS

1. Files:

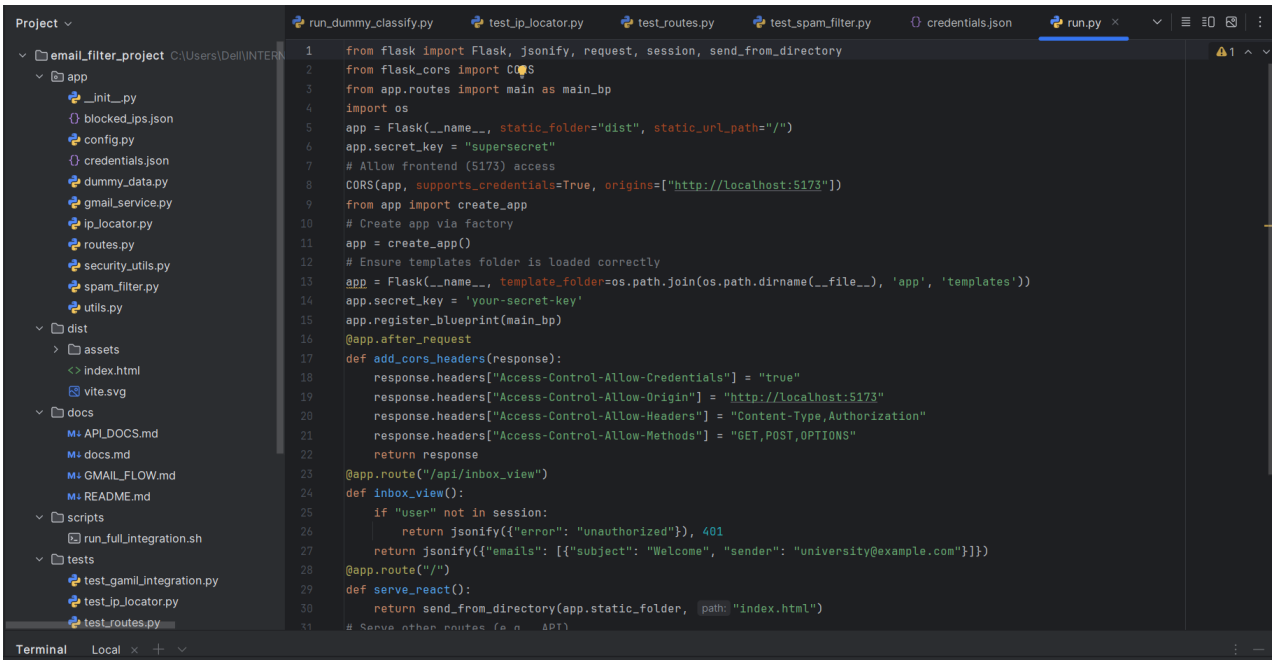

Figure 1:Run.py

This run.py file is the entry point of the Flask application, responsible for initializing and running the server.It configures CORS, registers routes, and connects the backend with the React frontend.

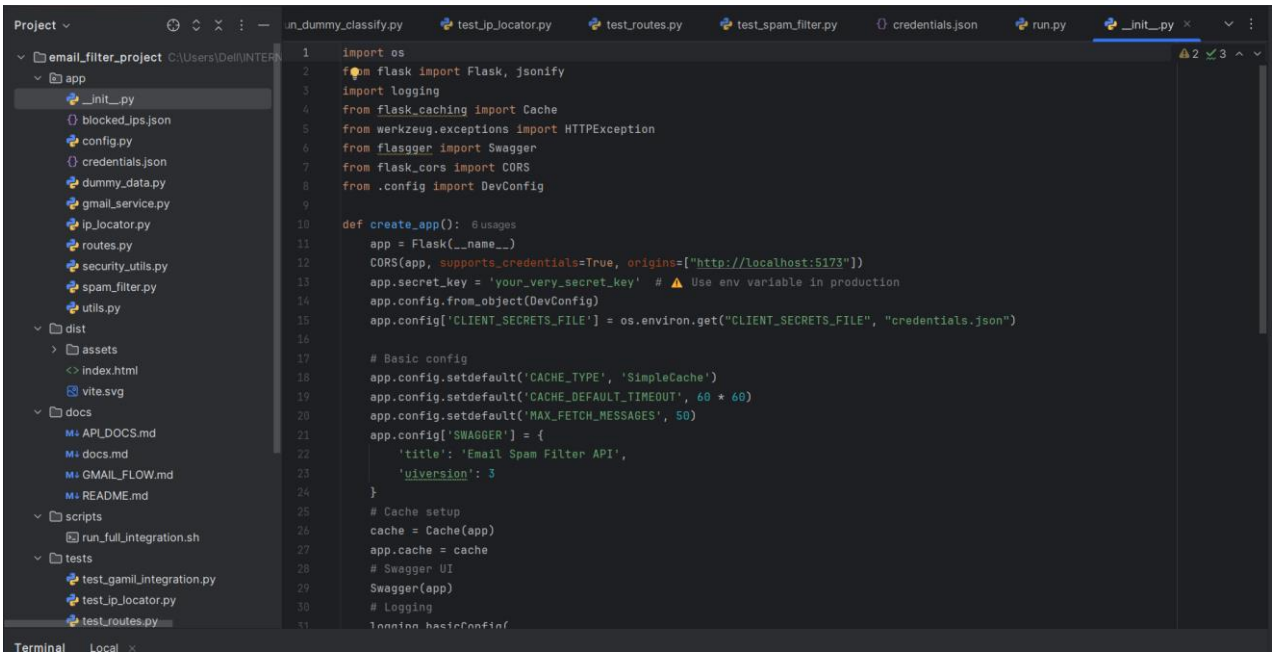

Figure 2: __init__.py

This init.py file initializes the Flask app using the factory pattern and configures caching, logging, and Swagger.It also registers API routes and defines global error handlers for the application.

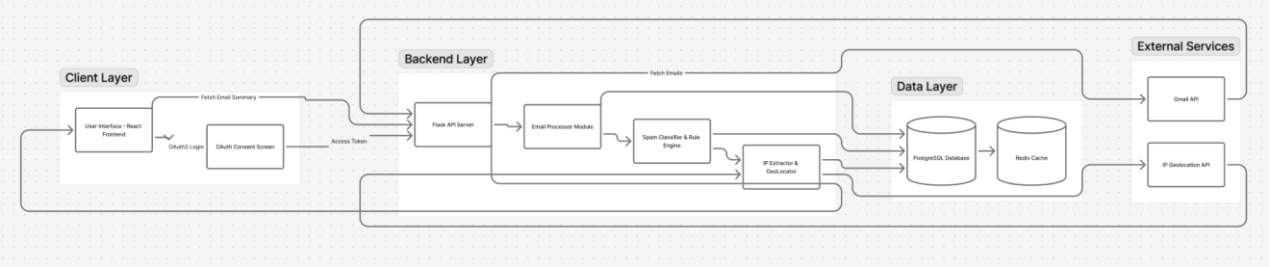

Figure 3: Architecture Diagram

This architecture diagram illustrates how emails flow securely from the Gmail API through Flask's spam classifier and IP tracker, then get stored and served to the frontend for user analysis.

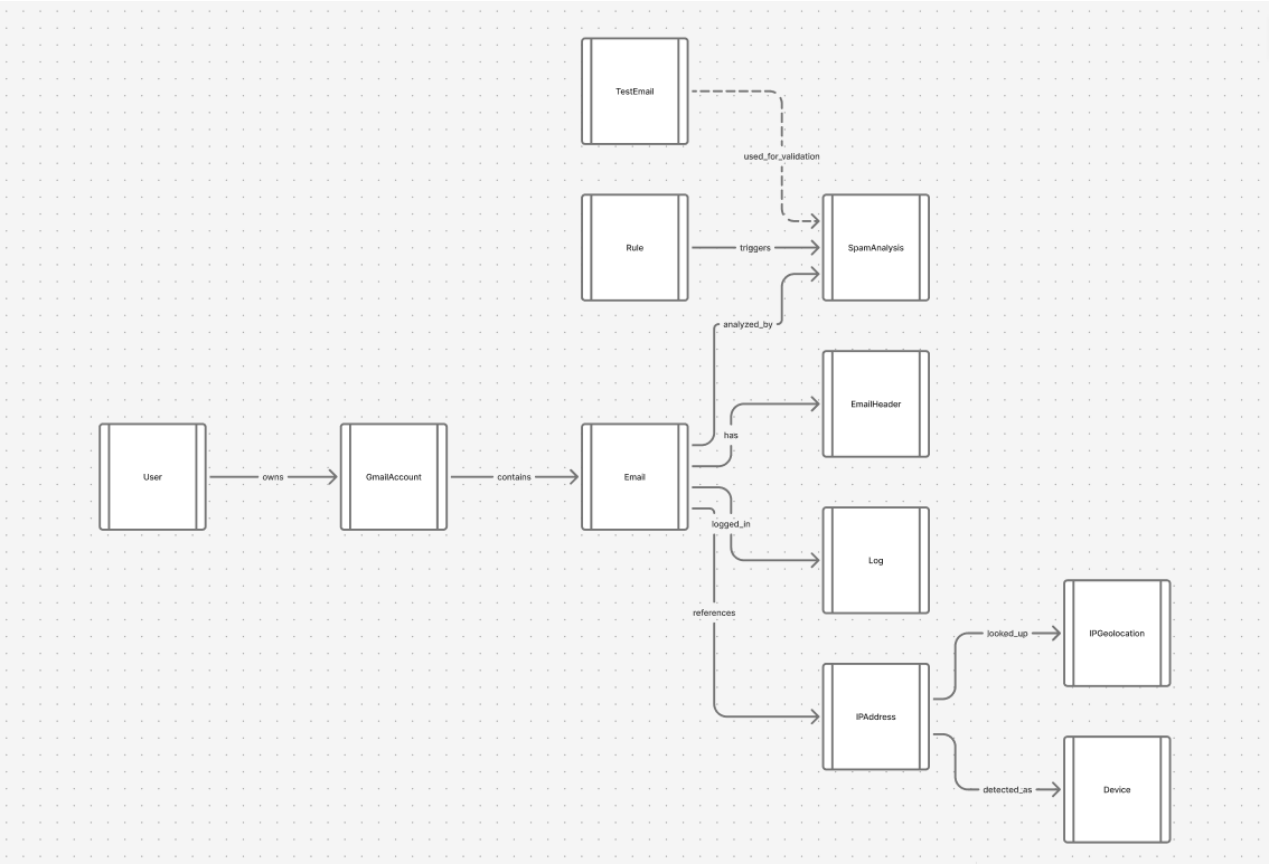

Figure 4: ER Diagram

The ER diagram shows how user emails are analyzed for spam using predefined rules and linked Gmail accounts.It tracks email headers, IP addresses, geolocation, and device details to identify and validate spam sources.

2. Results:

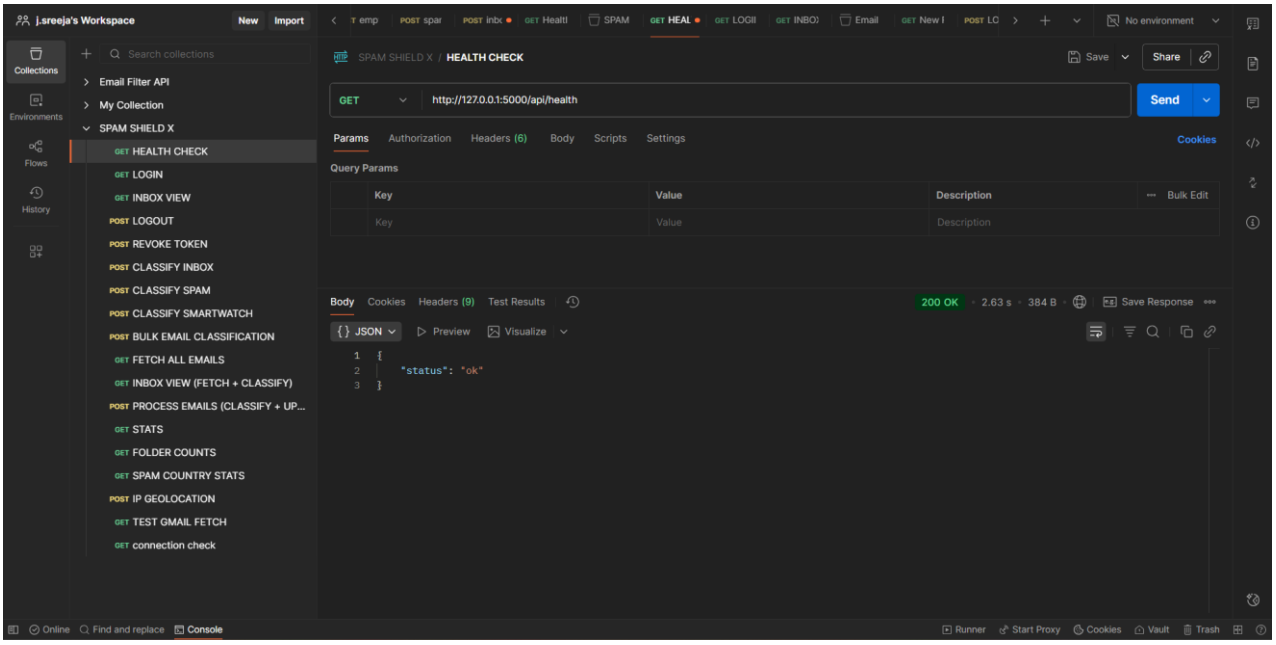

Figure 5: Health check (GET)

The figure 5 shows a Health Check API confirming that the Spam Shield X server is active and functioning correctly.

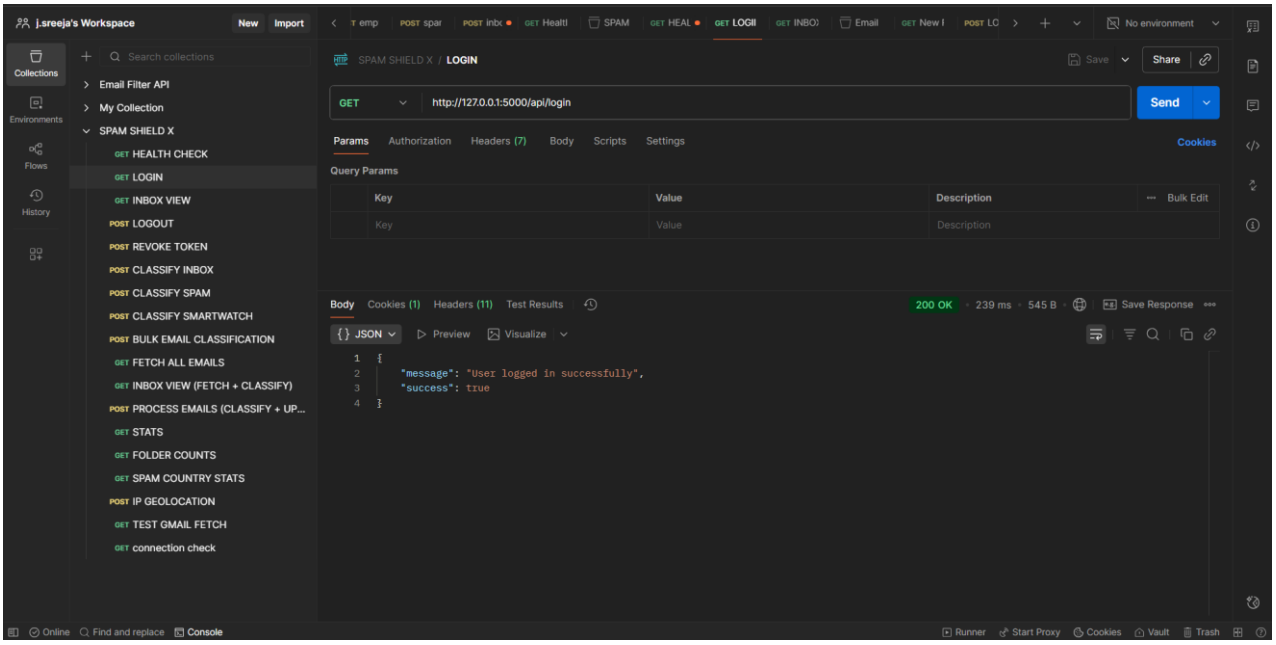

Figure 6: Login (GET)

The figure 6 shows a Login API confirming that the user has successfully logged into the Spam Shield X system.

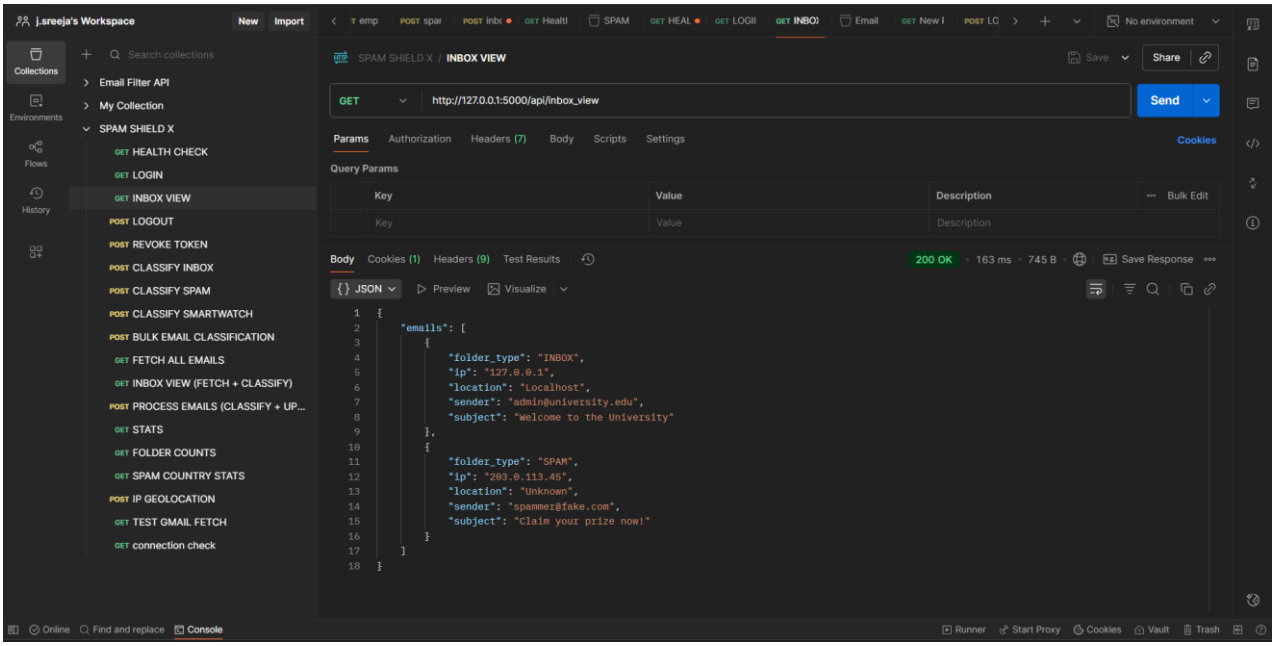

Figure 7: Inbox View (GET)

The figure 7 shows an Inbox View API displaying classified emails, separating university mails into Inbox and spam mails into Spam.

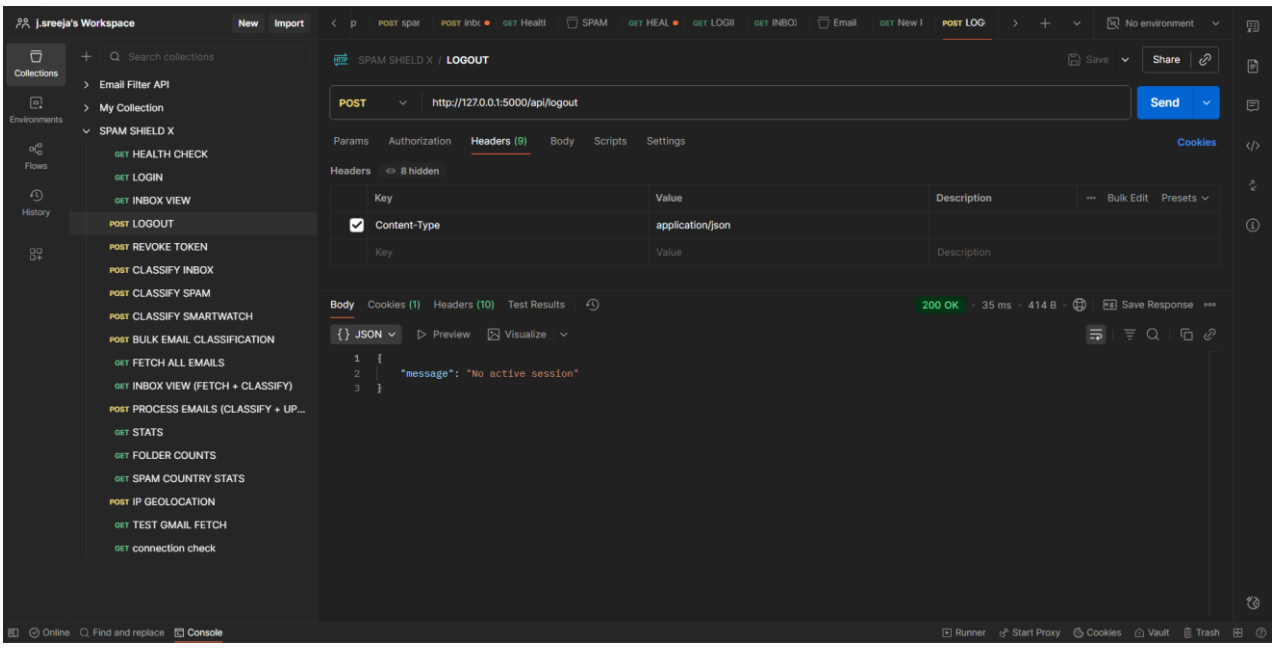

Figure 8: Logout (POST)

The figure 8 shows a Logout API indicating that there is no active user session in the Spam Shield X system.

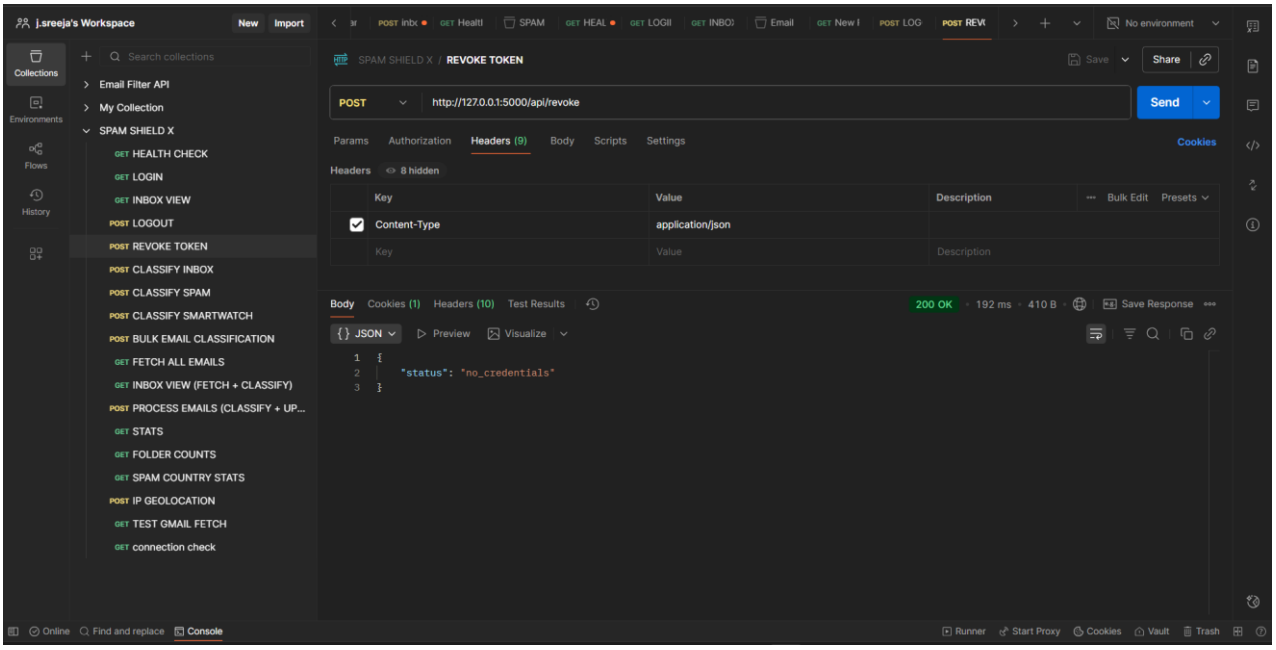

Figure 9: Revoke Token (POST)

The figure 9 shows a Revoke Token API indicating that no credentials are available to revoke in the Spam Shield X system.

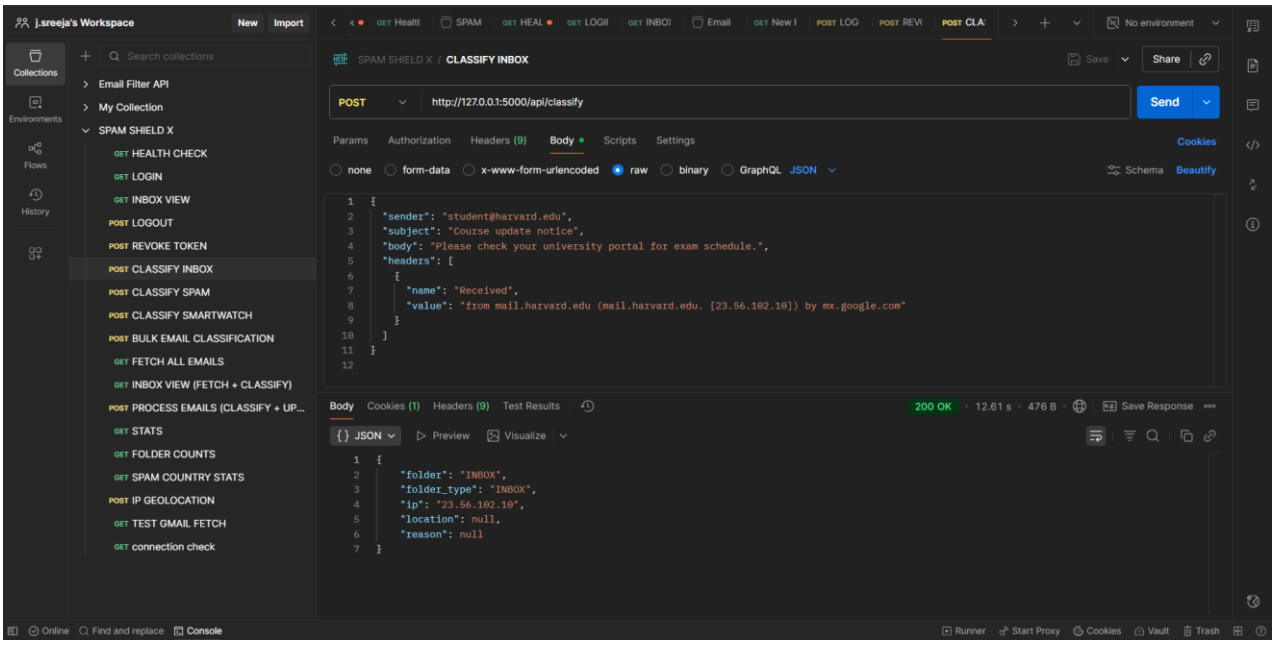

Figure 10: Inbox Classification (POST)

The figure 10 shows a Classify Inbox API that identifies a university-related email and categorizes it into the Inbox folder.

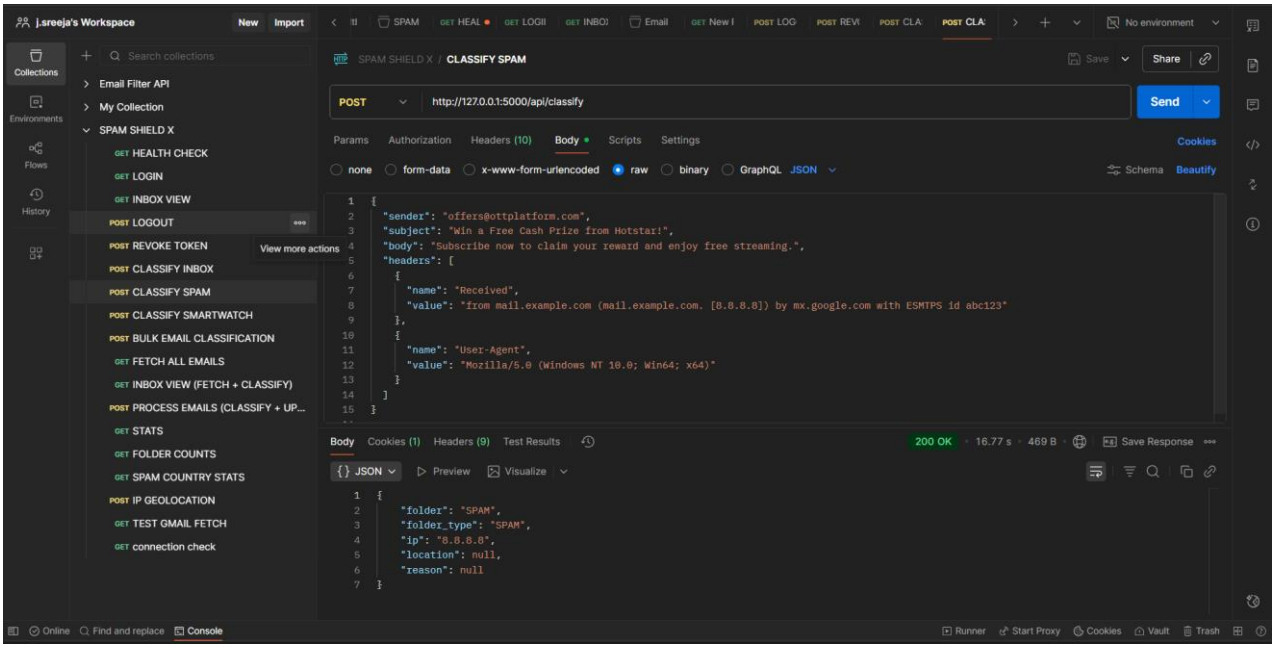

Figure 11: Spam Classification (POST)

The figure 10 shows a Classify Spam API that detects a promotional mail and categorizes it into the Spam folder.

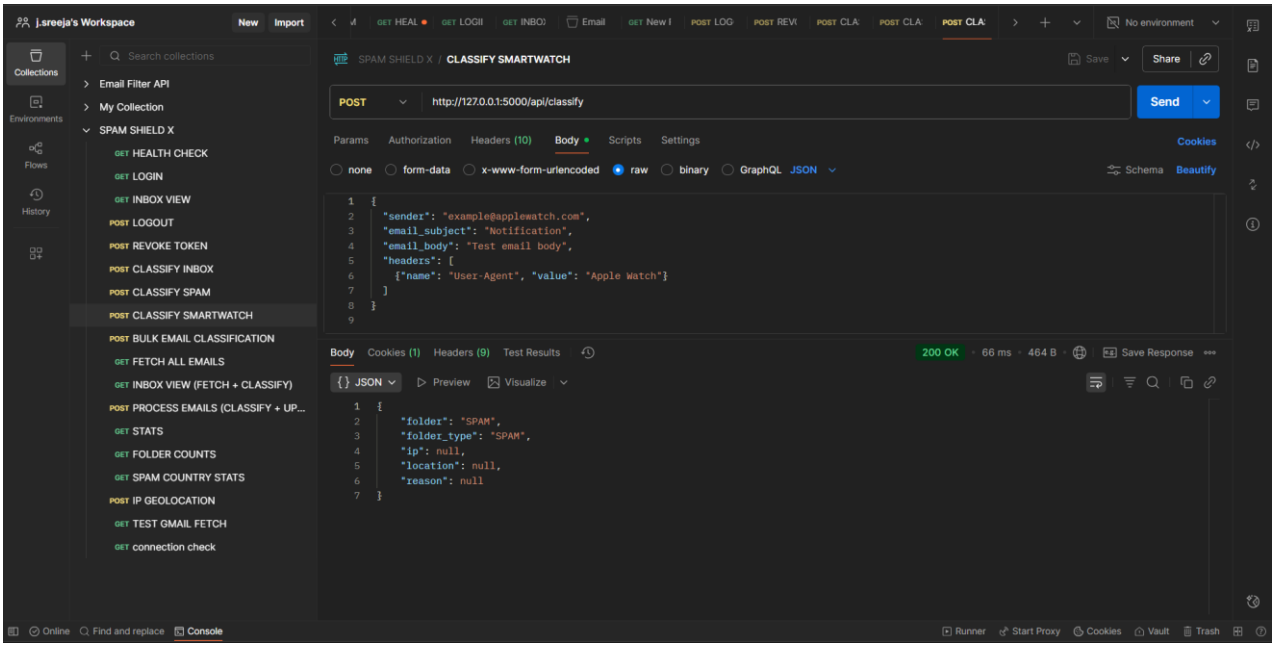

Figure 12: Spam Classification (POST)

The figure 11 shows a Classify Smartwatch API that detects an email sent from a smartwatch device and categorizes it as Spam.

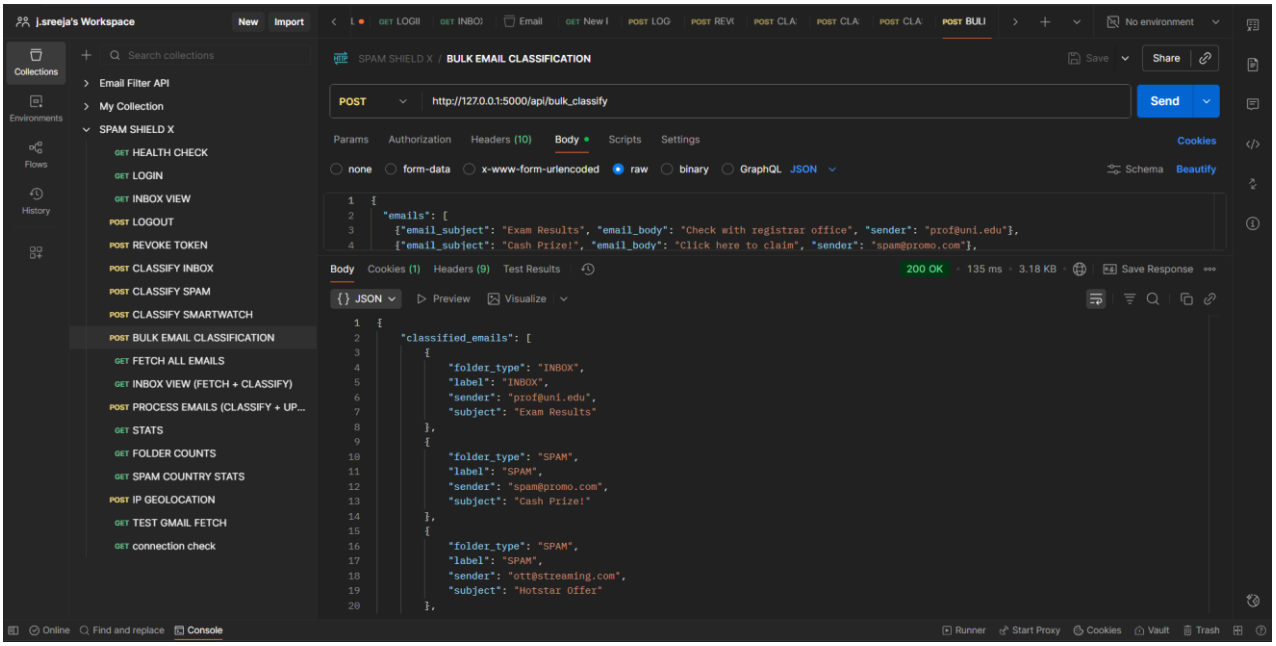

Figure 13: Bulk Email Classification (POST)

The figure 12 shows a Bulk Email Classification API that classifies multiple emails at once into Inbox and Spam categories.

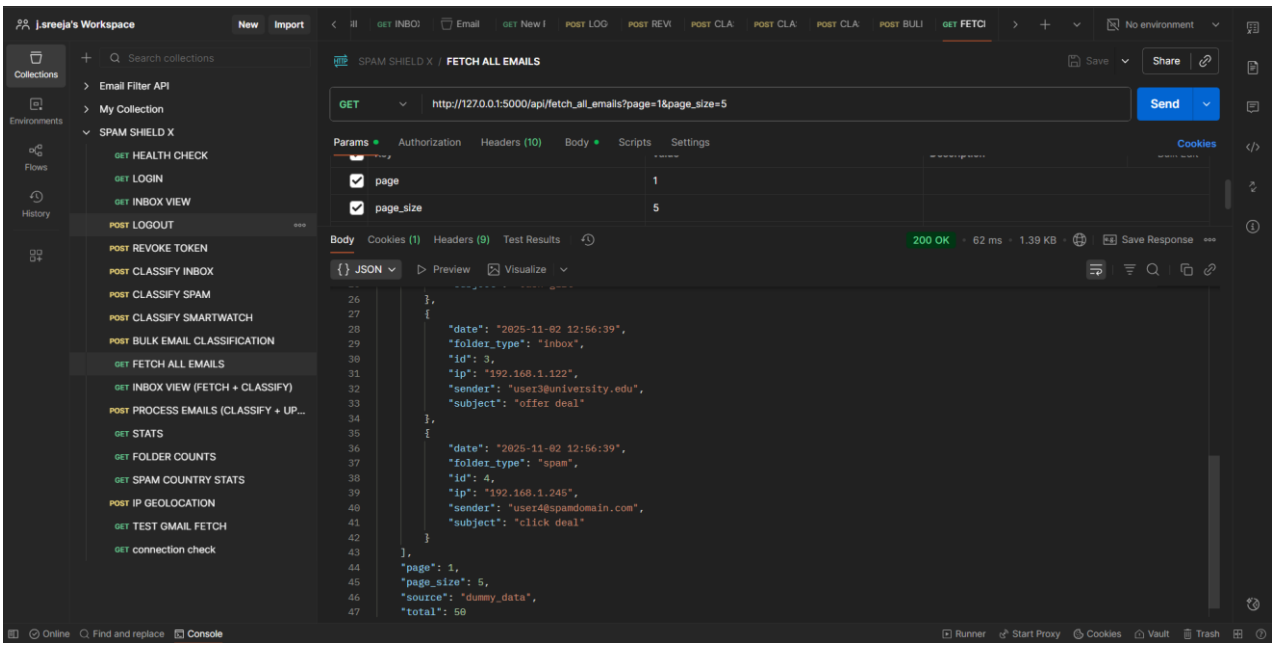

Figure 14: Fetch All Emails (GET)

The figure 13 shows a Fetch All Emails API that retrieves and displays all classified emails from both Inbox and Spam folders.

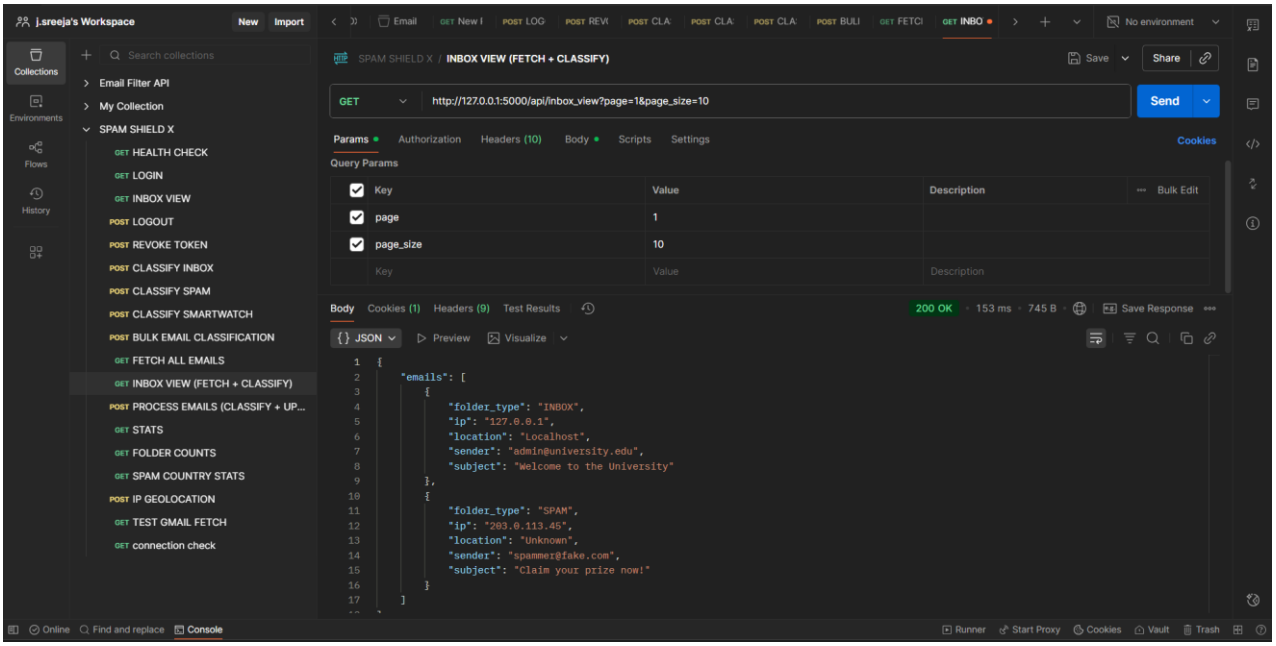

Figure 15: Inbox View (Fetch + Classify) (GET)

The figure 14 shows an Inbox View (Fetch + Classify) API that fetches emails and automatically classifies them into Inbox and Spam categories.

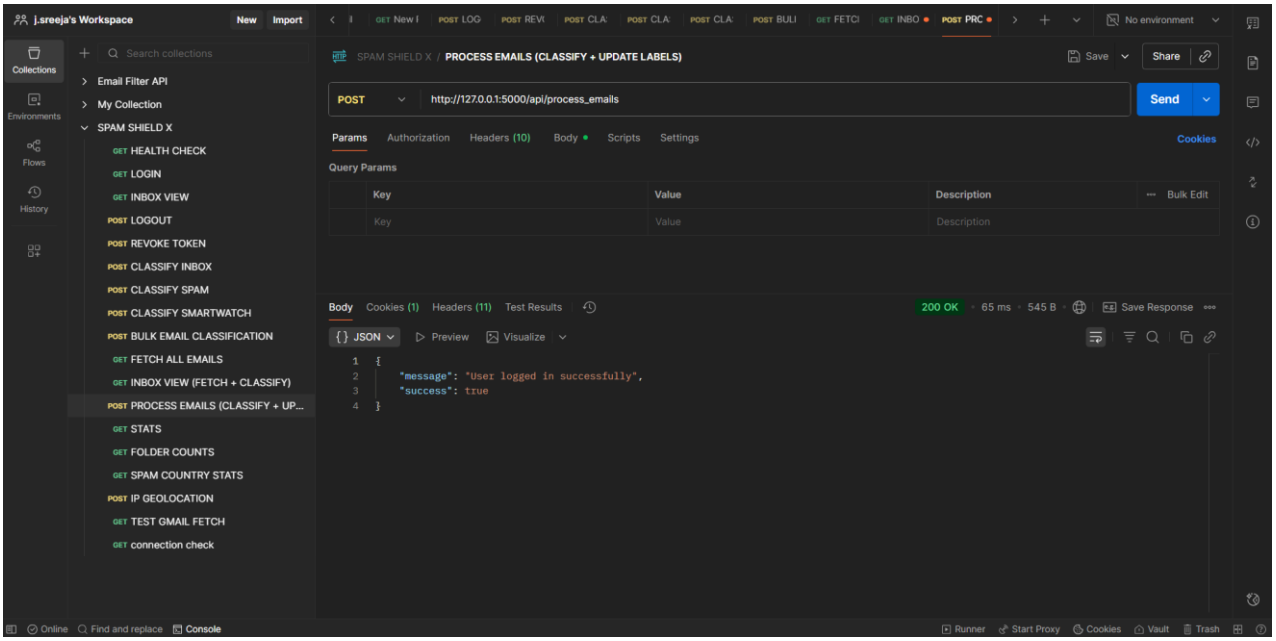

Figure 16: Process Emails (Classify + Update Labels) (POST)

The figure 16 shows a Process Emails API that classifies incoming mails and updates their labels after successful user authentication.

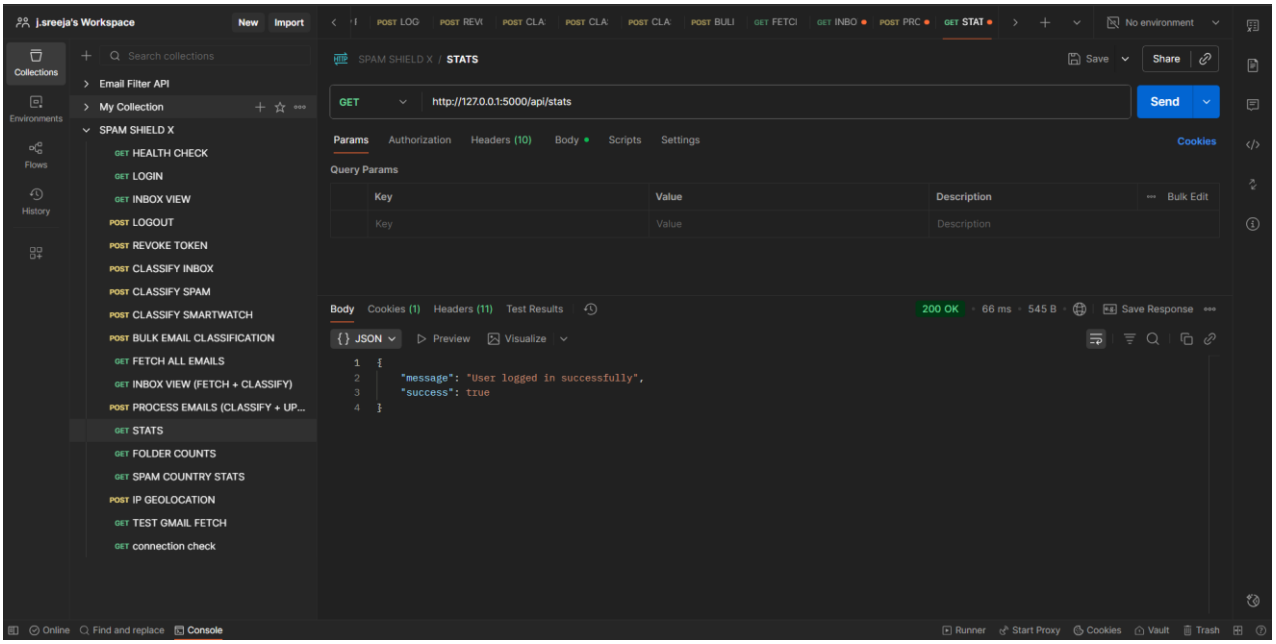

Figure 17: Stats (GET)

The figure 16 shows a Stats API that retrieves overall statistics of the Spam Shield X system after successful user authentication.

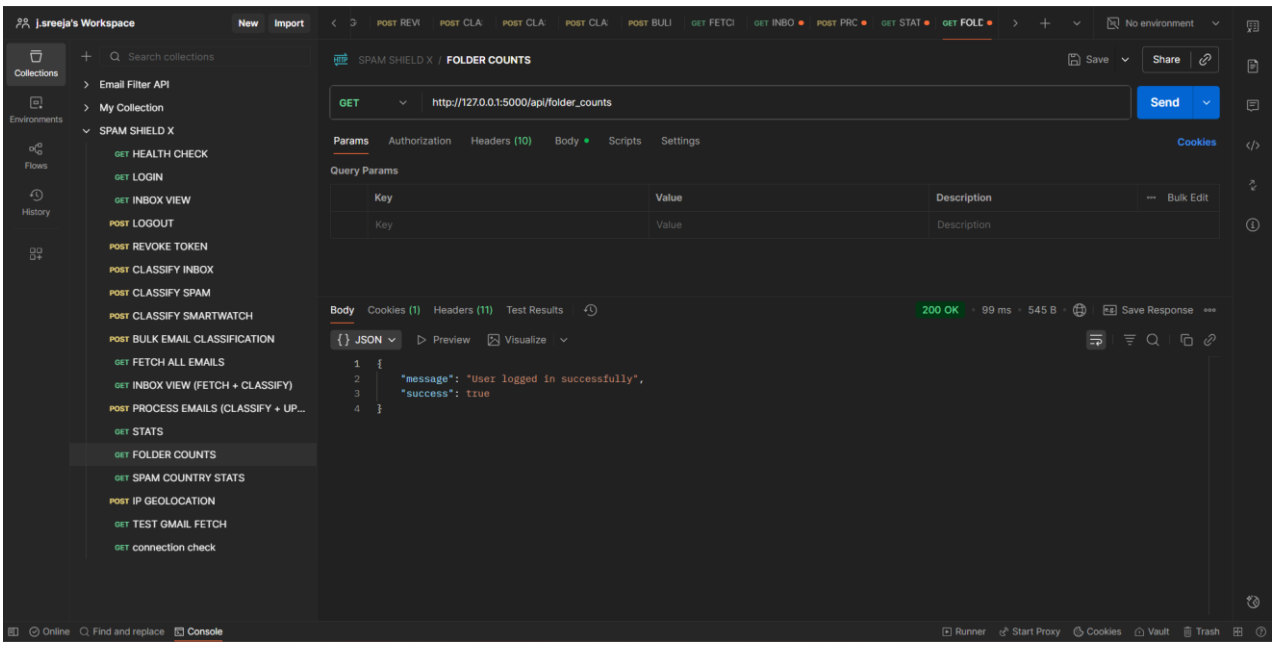

Figure 18: Folder Count (GET)

The figure 17 shows a Folder Counts API that retrieves the total number of emails in each folder after user authentication.

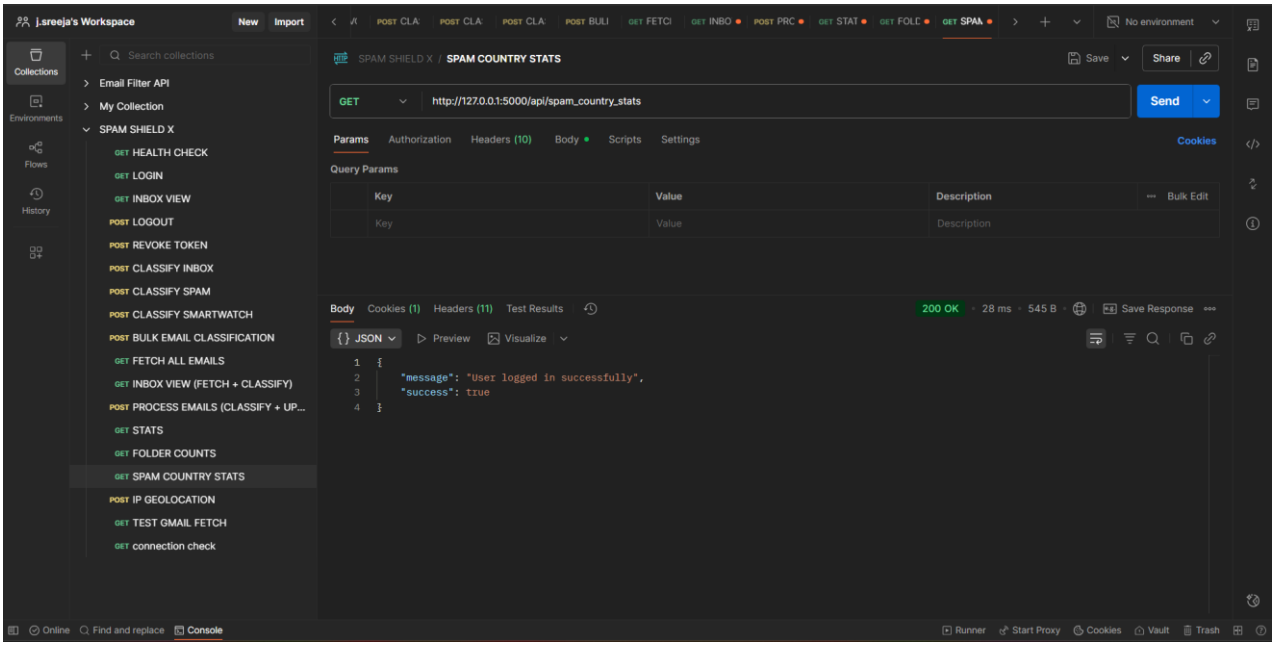

Figure 19: Spam Country Stats (GET)

The figure 18 shows a Spam Country Stats API that provides statistics of spam emails based on their country of origin.

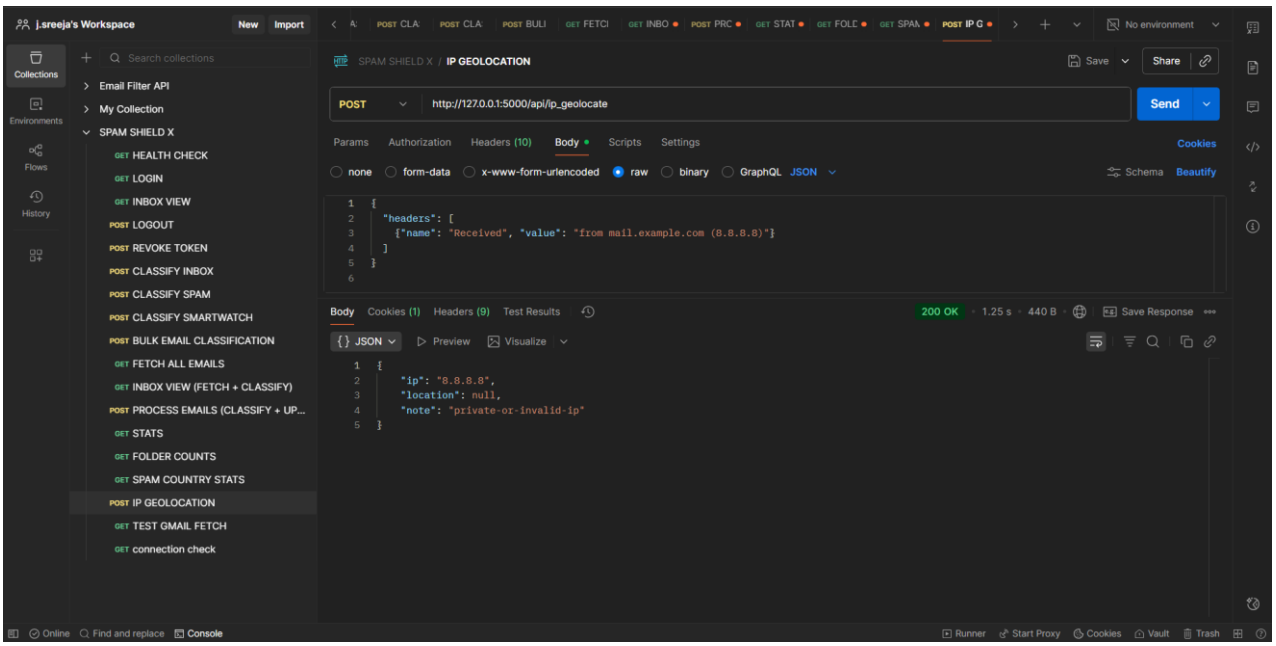

Figure 20: Ip Geolocation (POST)

The figure 19 shows an IP Geolocation API that traces the sender's IP address to identify the spammer's location.

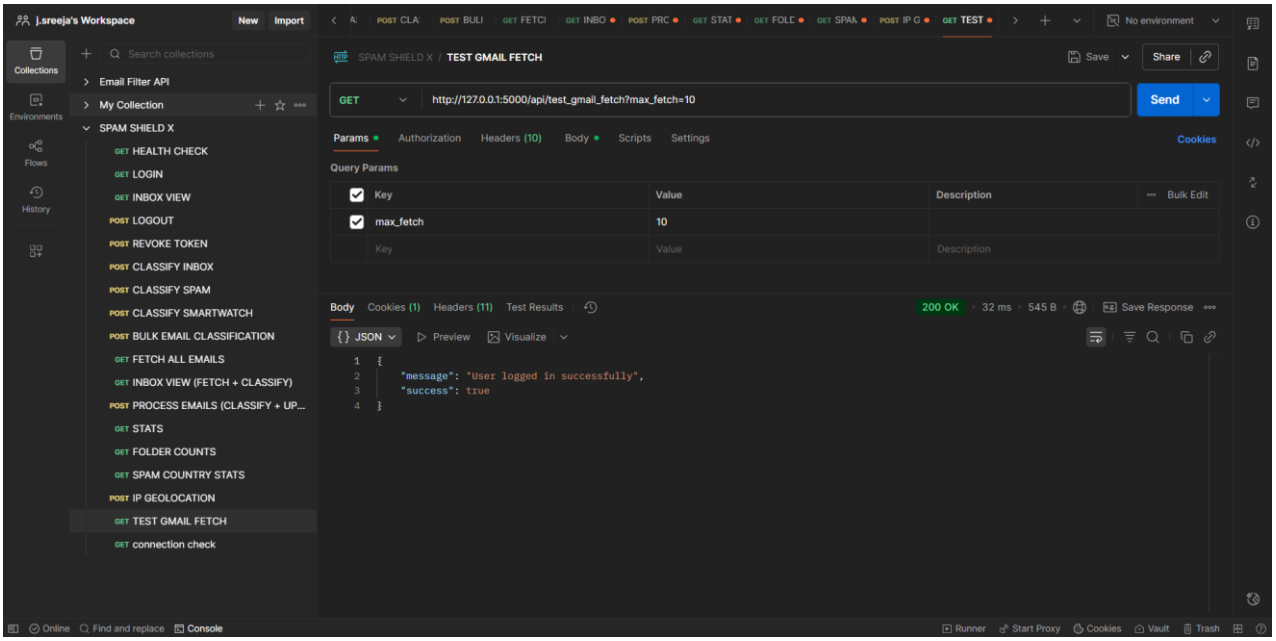

Figure 21: Test Gmail Fetch (GET)

The figure 20 shows a Test Gmail Fetch API that connects to Gmail and retrieves a specified number of recent emails for testing.

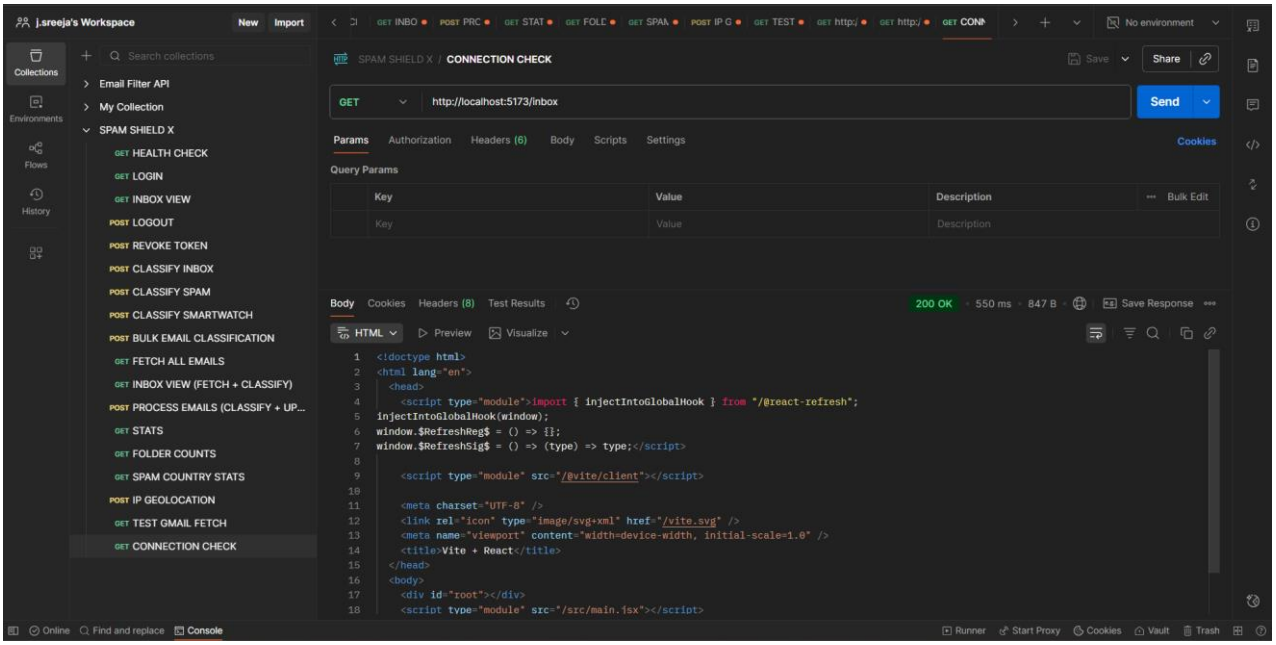

Figure 22: Connection Check (GET)

The figure 21 shows a Connection Check API that verifies the successful connection between the frontend and backend of the Spam Shield X system.

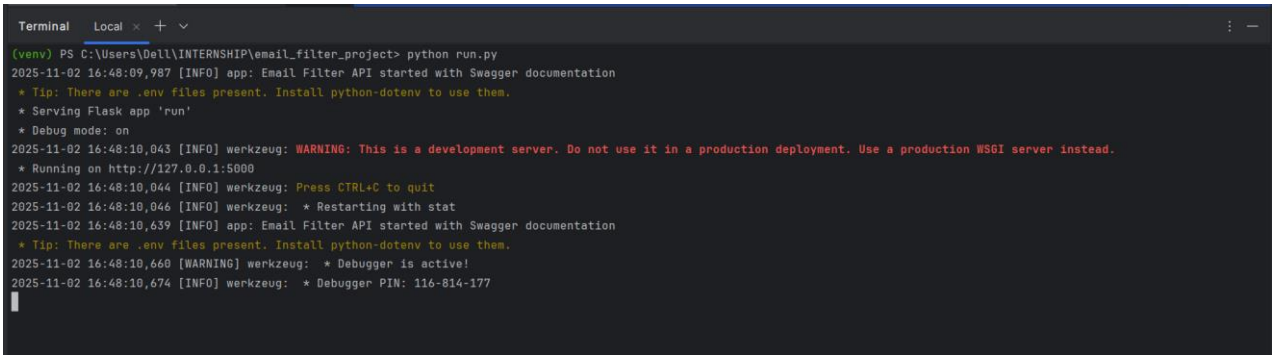

Figure 23: PyCharm Terminal

DISCUSSION

The goal of the backend development of the Spam Shield X was to develop a safe, automated method for identifying emails as spam or inbox and utilizing the IP address of the sender to determine their location. The primary goal was to make sure that all legitimate emails pertaining to the university were appropriately categorized into the Inbox and that any emails that were deemed irrelevant or suspicious—such as those about cash prizes, online offers, or unidentified platforms—were filtered into the Spam folder. In order to guarantee that only authentic emails from desktops, laptops, or mobile phones are accepted, the system was also made to recognize and block emails coming from smart devices like smartwatches.

The Flask project setup, which offered a lightweight and adaptable framework for backend development, was used to start the project. Flask was selected because of its ease of use and compatibility with APIs, which makes it appropriate for creating and overseeing numerous routes for IP analysis, email classification, and Gmail integration. In order to safely link the system with Gmail services, Gmail API OAuth credentials were then generated. To guarantee user authentication and authorization, the OAuth 2.0 flow was put into place, enabling the application to securely and privately access Gmail inbox data.

To automate the process of retrieving emails from the Gmail account, the Flask Gmail OAuth route setup and token flow implementation were finished after the authentication flow was established. The system was set up to analyze both spam and inbox emails. To decide whether an email should be categorized as spam or inbox, its metadata—which included headers, sender address, subject, and content—was processed. After that, the email classification logic was put into practice using predetermined guidelines and requirements, including domain name checks, the presence of dubious terms like "win," "cash," or "offer," and email source verification.

To guarantee classification accuracy, the outputs of the spam check were validated in the following stage. Using dummy emails that represented both authentic university messages and different types of spam communications, test cases were created to mimic real-world situations. This made it easier to confirm that the classification model was operating efficiently and reliably generating accurate results. The final classified emails could then be viewed or accessed via the proper Inbox or Spam endpoints thanks to the integration of the folder logic into Flask.

To determine the location of the original sender, a sizable portion of the backend involved extracting IP addresses and parsing email headers. The IP parser was improved to recognize and extract the correct originating IP address from complex header data because spam emails frequently go through multiple servers. The sender's country, region, and ISP information were then obtained by sending this IP to the ip-api geolocation service. Proper handling mechanisms were implemented for cases where IP lookups failed due to invalid or private addresses. To guarantee the accuracy of the geolocation results, the system was tested using a variety of IP sources from actual Gmail messages.

Features like folder-wise and country-wise spam counts were added to improve system intelligence. These features highlight the top countries from which spam emails are coming from and offer analytical insights into the percentage of spam and legitimate emails. Mechanisms for data filtering and pagination were added to effectively manage high email volumes without degrading performance. In order to gracefully handle API errors, null values, invalid tokens, and network failures, comprehensive error handling was also incorporated. This allowed for clear feedback for debugging and user guidance.

To automate the retrieval and batch classification of emails, the /fetch_all_emails API was subsequently developed. Email metadata, classification results, IP addresses, and geolocation information were among the structured data returned by this API. Code cleanup and utility refactoring were used to further optimize the backend, making it easier to read and maintain. For simpler frontend integration, documentation was added using Swagger and Markdown files, precisely outlining the available endpoints, request parameters, and response formats.

More than twenty dummy emails were used in extensive testing prior to final deployment in order to replicate a variety of scenarios, including spam ads, fraudulent offers, and university communications. Additionally, revoked tokens, IP blocking cases, and Gmail authentication failures were used to test the system's resilience. To guarantee stability and robustness, each of these cases was handled with grace. After that, a thorough integration test was carried out that covered every step of the process, from IP geolocation and classification to Gmail authorization and email fetching to result display.

The project's last phases were devoted to creating deliverables like the architecture diagram, ER diagram, and README documentation. The operational flow, module relationships, and backend design are summed up in these artifacts. To show how user authorization leads to data analysis, the Gmail integration flow was captured on video. Furthermore, folder-based summaries and country-specific spam analytics were verified to guarantee accurate data presentation and aggregation.

In conclusion, the backend successfully achieved its objectives by providing a robust framework capable of classifying emails, identifying spam sources through IP analysis, and preventing unauthorized smart-device-based emails. It is scalable for future improvements like AI-based spam detection models or sophisticated user dashboards because it guarantees data security, effective performance, and a clear modular design. A fully functional, tested, and documented backend system that reliably and precisely automates spam detection and sender location tracking is the end result.

CONCLUSION

The Flask framework was used to successfully develop the backend system for the "Spam Shield X" project, which integrated the Gmail API via secure OAuth authentication. By effectively dividing incoming emails into Inbox and Spam categories, the system makes sure that all official university emails end up in the inbox while detecting and separating unwanted emails, like spam or promotional messages. The backend offers precise email classification with high dependability by combining rule-based spam detection, keyword analysis, and sender verification.

The project uses geolocation and IP extraction to track the sender's origin in addition to spam identification, which aids in identifying possible spam sources. Spammer locations can be mapped globally thanks to the integration with IP-api, and the system intelligently blocks emails from unauthorized smart-watch IP addresses while permitting trusted devices like desktops, laptops, and mobile phones. Stability is ensured by robust error handling, logging, and testing, and insight into email traffic patterns is improved by the addition of analytics features like folder- and country-wise counts.

All things considered, the backend provides a comprehensive and safe email filtering pipeline, including data analytics, IP tracking, spam classification, and Gmail OAuth authentication. A dependable and useful method for detecting and stopping spam emails in actual communication settings, the project's scalability and maintainability are guaranteed by the use of testing, documentation, and visualization components (ER and architecture diagrams).

REFERENCES

1. Flask Documentation. (2024). Flask — Lightweight Python Web Framework.
2. Miguel Grinberg. (2022). Flask Web Development, 2nd Edition. O'Reilly Media. (Covers API design, OAuth, and logging in Flask)
3. Google Developers. (2024). Gmail API Overview.
4. Google Cloud. (2024). Using OAuth 2.0 for Web Server Applications.
5. Google API Python Client. (2024). Google API Python Client Library.
6. Sahami, M., Dumais, S., Heckerman, D., & Horvitz, E. (1998). A Bayesian Approach to Filtering Junk E-mail.AAAI Technical Report WS-98-05.
7. SpamAssassin Project. (2023). Apache SpamAssassin: Open Source Spam Filter.
8. Shanmugapriya, S., & Suresh, A. (2020). Email Spam Classification Using Machine Learning Techniques.International Journal of Scientific Research in Engineering and Management (IJSREM), Vol. 4, Issue 3.
9. IETF (Internet Engineering Task Force). (2008). RFC 5322 — Internet Message Format.
10. Python Email Package. (2024). Email: parsing email messages.
11. GeeksforGeeks. (2023). Extracting IP Address from Email Header using Python.
12. IP-API Documentation. (2024). IP Geolocation API — Detect Location from IP Address.
13. MaxMind. (2024). IP Geolocation and Online Fraud Prevention.
14. Python Official Documentation. (2024). unittest — Unit Testing Framework.
15. Real Python. (2023). Logging in Python — How To Log Messages Effectively.
16. Flask Testing Patterns. (2023). Testing Flask Applications.
17. Swagger.io. (2024). OpenAPI Specification (Swagger) for API Documentation.
18. RESTful API Design Guidelines. (2024). Best Practices for Designing Flask APIs.
19. OWASP Foundation. (2024). Top 10 Security Risks for Web Applications.
20. Twelve-Factor App. (2023). Configuration Best Practices for Flask Apps.
21. Pandas Documentation. (2024). Pandas: Data Analysis Library.
22. Matplotlib Documentation. (2024). Data Visualization in Python.
23. Chart.js. (2024). Frontend Visualization Framework.
24. Bassiouni, M., & Hassan, M. (2021). An Improved Email Spam Detection Model Based on Machine Learning Techniques.Journal of Computer Networks and Communications, Volume 2021, Article ID 6695728.

25. Jain, A., & Sharma, P. (2022). Email Spam Detection Using NLP and Machine Learning.International Journal of Innovative Research in Computer Science & Technology (IJIRCST), Vol. 10, Issue 4.
26. Lucidchart. (2024). ER Diagram & Architecture Diagram Design Tool.
27. Draw.io. (2024). Online Tool for Flowcharts and Architecture Diagrams.